

Vulnerability Assessment Report

Cyber Security Internship — Future Interns

Intern: Lalas Maruthi | CIN ID: FIT/AUG26/CS10164

Target: testphp.vulnweb.com (Acunetix's publicly maintained test site for security-testing practice)

Date: 25-08-2026

1. Why I chose this target

For this task I needed a website I was actually allowed to test. Rather than pick a random live site, I used testphp.vulnweb.com, which Acunetix runs specifically so people learning security can practice on it legally. I wanted my assessment to be something I could stand behind, not something built on a target I had no authorization to touch.

2. What I actually did

I started with Nmap to see what was running on the host, then used OWASP ZAP in passive mode to crawl the site without sending any attack payloads. After the scan finished, I went back through the flagged pages manually in the browser with DevTools open, so I wasn't just trusting the tool's output — I wanted to confirm each finding myself before writing it down.

- Nmap — to check open ports and identify the web server/technology stack
- OWASP ZAP (passive scan) — to crawl the app and flag likely issues without exploiting anything
- Browser DevTools — to manually check headers, cookies, and request/response behavior
- Canva — for laying out the final client-facing version of this report

3. Summary of what I found

Overall, I'd rate this site as HIGH risk if it were a real production application. It has two serious issues, a couple of medium ones around configuration, and one minor cookie-handling gap.

ID	Finding	Risk	Ref (CWE)
V-01	SQL Injection in the login/search form	High	CWE-89
V-02	Reflected Cross-Site Scripting (XSS)	High	CWE-79
V-03	Missing security headers (CSP, X-Frame-Options, HSTS)	Medium	CWE-693
V-04	Server banner reveals version information	Medium	CWE-200
V-05	Session cookies missing Secure/HttpOnly flags	Low	CWE-614

4. Findings, explained the way I'd explain them to a client

V-01: SQL Injection (High)

This was the one that stood out most. A form on the site takes what you type and drops it straight into a database query without cleaning it first. In plain terms — if this were a real business, someone could use this weakness to read or tamper with customer data they have no business seeing.

What I'd tell them to do:

- Rewrite the query logic to use parameterized queries instead of building queries from raw input
- Add strict input validation on the server side, not just in the browser
- A web application firewall can buy time while the actual fix is being developed

V-02: Reflected XSS (High)

Some pages take a value like a search term and print it back onto the page without cleaning it up. That means someone could craft a link that runs their own code in another visitor's browser when clicked — which could be used to hijack a session.

- Encode output properly depending on where it's being inserted (HTML vs. JavaScript vs. URL)
- Add a Content-Security-Policy header to limit what scripts are allowed to run

V-03: Missing Security Headers (Medium)

The site doesn't send a few headers that modern browsers use to enforce extra protection — things like blocking the page from being embedded in someone else's frame, or forcing HTTPS. None of these alone is catastrophic, but leaving them off makes other attacks easier.

- Add Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Strict-Transport-Security

V-04: Server Version Disclosure (Medium)

The server response gives away the exact software version it's running. That's useful to an attacker because they can just look up known vulnerabilities for that specific version instead of guessing.

- Strip or generalize version info from response headers, and keep patching on schedule regardless

V-05: Cookie Flags Missing (Low)

Small thing, but worth flagging — session cookies aren't marked Secure or HttpOnly, so they could theoretically be intercepted or accessed by a malicious script.

- Set Secure, HttpOnly, and SameSite on every session cookie

5. Challenges I ran into

Honestly, the hardest part wasn't finding the issues — ZAP flags most of them automatically. The harder part was resisting the urge to just copy the scanner's output verbatim. I went back and manually checked each flagged item in the browser to make sure I actually understood why it was a problem, not just that a tool said so. That took longer, but it's the part that actually taught me something.

6. What I'd prioritize if this were real

- **Immediately: fix SQL Injection and XSS — these are the ones that could cause real damage**
- Within a few weeks: add the missing security headers and hide version info
- Ongoing: cookie hardening and periodic re-scans as part of normal maintenance

7. Closing thoughts

This task gave me a much better feel for what a real vulnerability assessment looks like end to end — not just running a scanner, but verifying results.